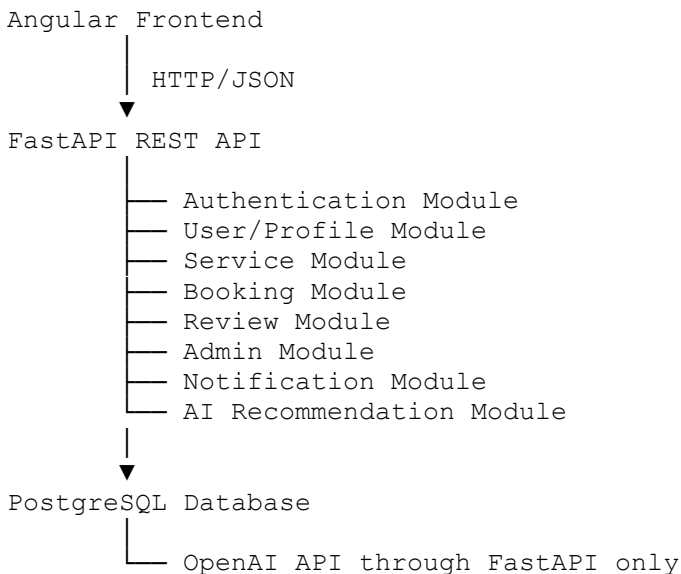


Based on your assignment, SRS, database design, WBS, use cases, user stories, and API documentation, the easiest approach is to build **Smart Hire as a modular monolithic application** rather than using microservices.

The assignment permits FastAPI or Spring Boot for the backend, Angular or HTML/CSS/JavaScript for the frontend, and PostgreSQL or MySQL for the database. Your existing documents already mainly use **Angular + FastAPI + PostgreSQL**, so the team should continue with that stack to avoid rewriting documentation.

1. Recommended easy architecture



Use a **three-tier architecture**:

1. Presentation layer – Angular
2. Application layer – FastAPI
3. Data layer – PostgreSQL

The OpenAI API key must remain in the FastAPI backend environment variables and must never be placed in Angular. This matches your API documentation.

Why this architecture is suitable

- Easy for five students to understand.
- Each member can complete both frontend and backend work.
- Only one backend application and one frontend application.
- Easier deployment than microservices.
- Shared authentication and database connection.
- FastAPI automatically generates Swagger documentation.
- Modules can be developed separately using Git branches.

Do not build separate microservices for users, bookings, services, and AI. That would add unnecessary deployment, communication, authentication, and debugging complexity.

2. Final technology stack

Use one fixed stack across the project:

Layer	Technology
Frontend	Angular
UI styling	Angular Material or Bootstrap
Backend	Python FastAPI
ORM	SQLAlchemy
Validation	Pydantic
Database	PostgreSQL
Migration	Alembic
Authentication	JWT and bcrypt
AI	OpenAI API
API testing	Postman
Backend testing	Pytest
Version control	GitHub
Project management	Jira

Your WBS currently mentions HTML, CSS and JavaScript in some sections, while the testing and API documents mention Angular. Select **Angular** officially and update all documents so that the technology stack is consistent.

3. Divide the system into five vertical modules

Because every member must do frontend and backend work, do not assign one member only frontend and another only backend.

Each member should own a complete vertical feature:

```
Database tables
↓
FastAPI models and endpoints
↓
Angular pages and services
↓
Testing
↓
Documentation
```

Member 1 – Authentication and User Profiles

Frontend

- Customer registration page
- Provider registration page
- Login page
- Profile page
- Edit-profile form
- Role-based navigation
- Route guards

Backend

- User registration
- Login
- Password hashing
- JWT token generation
- Current-user endpoint
- Profile update
- Role-based access control

Database tables

- users
- provider_profiles

Main API endpoints

POST /api/auth/register
POST /api/auth/login
GET /api/auth/me
PUT /api/users/profile
GET /api/providers/{provider_id}
PUT /api/providers/profile

Tests

- Registration with valid data
- Duplicate email prevention
- Valid and invalid login
- Unauthorized access
- Role-based access

This module covers FR1–FR6 from the SRS.

Member 2 – Service Catalog and Search

Frontend

- Home page
- Category listing
- Service search page
- Filters
- Service detail page
- Provider service-management page
- Add-service form
- Edit-service form
- Favourite-services page

Backend

- Category listing
- Create, update and delete services
- Search by keyword
- Filter by category
- Filter by location
- Retrieve service details
- Save and remove favourites

Database tables

- service_categories
- services
- service_images
- service_areas
- favorites

Main API endpoints

```
GET    /api/categories
GET    /api/services
GET    /api/services/{service_id}
POST   /api/services
PUT    /api/services/{service_id}
DELETE /api/services/{service_id}
GET    /api/services/search
POST   /api/favorites/{service_id}
DELETE /api/favorites/{service_id}
GET    /api/favorites
```

Tests

- Provider creates a service
- Customer searches by keyword
- Category filtering
- Provider edits only their own services
- Customer saves and removes favourites

The database design already includes service categories, services, images, service areas and favourites.

Member 3 – Booking, Scheduling and Notifications

Frontend

- Booking form
- Select date and time
- Customer booking history
- Customer booking detail
- Provider booking-request page
- Accept/reject buttons
- Mark as completed button
- Notifications page

Backend

- Create booking
- View customer bookings
- View provider booking requests
- Accept booking
- Reject booking
- Cancel booking
- Complete booking
- Create booking notifications
- Retrieve and mark notifications as read

Database tables

- bookings
- provider_availability
- notifications

Main API endpoints

```
POST /api/bookings
GET /api/bookings/customer
GET /api/bookings/provider
GET /api/bookings/{booking_id}
PUT /api/bookings/{booking_id}/accept
PUT /api/bookings/{booking_id}/reject
PUT /api/bookings/{booking_id}/cancel
PUT /api/bookings/{booking_id}/complete

GET /api/notifications
PUT /api/notifications/{notification_id}/read
```

Booking statuses

Use only these values:

```
pending
accepted
rejected
in_progress
```

completed
cancelled

Important business rules

- Only a customer can create a booking.
- Only the related provider can accept or reject it.
- A customer can cancel only a pending or accepted booking.
- Only completed bookings can receive reviews.
- Every important status change creates a notification.

The SRS specifies booking creation, viewing, acceptance, rejection, cancellation and status tracking.

Member 4 – Reviews, Payments and Admin

Frontend

- Submit-review form
- Provider review list
- Rating display
- Payment-status form
- Admin dashboard
- User-management table
- Category-management page
- Service-moderation page
- Booking-monitoring page

Backend

- Submit a review
- Retrieve provider reviews
- Calculate average rating
- Record payment
- View payment status
- Admin user management
- Admin category management
- Admin service moderation
- Admin statistics

Database tables

- reviews
- payments
- Admin operations use:
 - users
 - services
 - service_categories
 - bookings

Main API endpoints

```
POST /api/reviews
GET /api/providers/{provider_id}/reviews

POST /api/payments
GET /api/payments/booking/{booking_id}

GET /api/admin/dashboard
GET /api/admin/users
PUT /api/admin/users/{user_id}/status
POST /api/admin/categories
PUT /api/admin/categories/{category_id}
DELETE /api/admin/categories/{category_id}
DELETE /api/admin/services/{service_id}
GET /api/admin/bookings
```

Payment simplification

For the university project, do not integrate a real payment gateway unless explicitly required.

Use a payment-recording flow:

Customer chooses:

- Cash
- Card - simulated
- Bank transfer - simulated

System stores:

- payment method
- amount
- payment status
- transaction reference

Clearly state that the payment is simulated for academic demonstration.

The database document already defines a one-to-one payment record for each booking and one review per completed booking.

Member 5 – AI Assistant and Recommendations

Frontend

- AI assistant chat page
- Problem-description input
- Recommended-category result
- Recommended-provider cards
- Review-summary display
- Chat history

Backend

- AI recommendation endpoint
- Retrieve valid service categories from the database
- Create a controlled prompt
- Call OpenAI
- Validate the returned category
- Find providers from PostgreSQL
- Generate review summaries
- Store AI conversations and messages

Database tables

- ai_conversations
- ai_messages
- review_summaries

For the first version, `service_embeddings` and `pgvector` can be postponed. Start with ordinary PostgreSQL filtering.

Main API endpoints

```
POST /api/ai/recommend
POST /api/ai/provider-match
POST /api/ai/reviews/summarize
POST /api/ai/chat
GET  /api/ai/conversations
GET  /api/ai/conversations/{conversation_id}
```

Easy AI implementation

The simplest useful AI feature is:

```
Customer:
"My kitchen sink is leaking."
```

```
AI:
Recommended category: Plumbing
Reason: The request describes a water leakage problem.
```

Then the backend searches the normal services table:

```
SELECT *  
FROM services  
WHERE category_id = :recommended_category_id  
AND status = 'active';
```

This is easier and more dependable than building embeddings first.

Your assignment requires at least one AI-powered feature, while the documentation currently proposes service recommendation, provider matching, customer support and review summarization. Completing one feature properly is more valuable than implementing four incomplete AI features.

4. Shared responsibilities

Although each member owns one module, everyone should contribute to these areas:

Shared task	Responsibility
Git commits	Every member
Jira updates	Every member
Unit tests	Module owner
API tests	Module owner
Frontend validation	Module owner
Documentation updates	Module owner
Code review	Another member
Viva preparation	Every member
Integration testing	Whole team
Final deployment	Whole team

The assignment gives separate marks for Git, project management, development, database design and QA. Therefore, every member should leave visible evidence in commits, Jira tasks, tests and documentation.

5. Suggested project structure

FastAPI backend

```
backend/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── security.py
│   │   └── dependencies.py
│   ├── models/
│   │   ├── user.py
│   │   ├── provider.py
│   │   ├── service.py
│   │   ├── booking.py
│   │   ├── review.py
│   │   ├── payment.py
│   │   ├── notification.py
│   │   └── ai.py
│   ├── schemas/
│   │   ├── auth.py
│   │   ├── user.py
│   │   ├── service.py
│   │   ├── booking.py
│   │   ├── review.py
│   │   ├── payment.py
│   │   └── ai.py
│   └── routers/
```

```
├── auth.py
├── users.py
├── providers.py
├── services.py
├── bookings.py
├── reviews.py
├── payments.py
├── notifications.py
├── admin.py
├── ai.py
├── services/
│   ├── auth_service.py
│   ├── booking_service.py
│   ├── notification_service.py
│   └── ai_service.py
├── tests/
│   ├── test_auth.py
│   ├── test_services.py
│   ├── test_bookings.py
│   ├── test_reviews.py
│   └── test_ai.py
├── alembic/
├── requirements.txt
├── .env.example
└── README.md
```

Angular frontend

```
frontend/src/app/
├── core/
│   ├── guards/
│   ├── interceptors/
│   ├── services/
│   └── models/
├── shared/
│   ├── components/
│   └── layouts/
├── features/
│   ├── auth/
│   ├── profile/
│   ├── services/
│   ├── bookings/
│   ├── reviews/
│   ├── payments/
│   ├── notifications/
│   ├── admin/
│   └── ai-assistant/
├── app.routes.ts
└── app.component.ts
```

6. Important database decision

Your uploaded class diagram is simpler than the detailed database design. The class diagram shows entities such as User, ProviderProfile, Service, Booking, Review, Favourite, Category, Notification and AIConversation.

The formal database document expands this into sixteen tables, including:

- service images
- service areas
- provider availability
- payments
- AI messages
- review summaries
- embeddings

For development, use two scopes.

MVP tables

Build these first:

```
users
provider_profiles
service_categories
services
bookings
reviews
favorites
notifications
ai_conversations
ai_messages
```

Optional tables after MVP

```
service_images
service_areas
provider_availability
payments
review_summaries
service_embeddings
```

This prevents the project from becoming too large before the submission date.

7. Suggested development order

Sprint 1 – Foundation

- Create GitHub repository.
- Create Jira board.
- Set up FastAPI.
- Set up Angular.
- Set up PostgreSQL.
- Configure SQLAlchemy.

- Create migrations.
- Complete authentication.
- Build shared navbar and layouts.

Sprint 2 – Main marketplace

- Provider profile
- Categories
- Service creation
- Service search
- Service detail
- Customer and provider dashboards

Sprint 3 – Booking flow

- Create booking
- Provider accept/reject
- Customer cancel
- Complete booking
- Notifications
- Booking history

Sprint 4 – Reviews, admin and AI

- Reviews and ratings
- Admin dashboard
- User and category management
- AI service recommendation
- AI provider matching

Sprint 5 – Testing and deployment

- Unit tests
- Postman tests
- Integration tests
- Bug fixing
- Responsive design
- Deployment
- Documentation evidence
- Viva preparation

8. Git branching strategy

Use one branch for each member's module:

```
main
develop
feature/auth-profile
feature/service-catalog
```

feature/booking-notifications
feature/review-admin
feature/ai-assistant

Workflow:

```
feature branch
  ↓
Pull request to develop
  ↓
Another member reviews it
  ↓
Merge into develop
  ↓
Test complete application
  ↓
Merge develop into main
```

Each commit should be clear:

```
feat(auth): add JWT login endpoint
feat(services): add service creation form
fix(bookings): prevent cancelling completed bookings
test(ai): add recommendation endpoint tests
docs(api): document booking status endpoint
```

Do not let one member upload all code at the end. That will make individual contribution difficult to prove during the viva.

9. Ownership matrix

Member	Primary module	Secondary responsibility
Member 1	Authentication and profiles	Backend structure and security
Member 2	Services and search	Shared Angular UI components
Member 3	Bookings and notifications	Integration coordinator
Member 4	Reviews, payments and admin	QA coordinator
Member 5	AI features	Deployment and environment setup

Secondary responsibility does not mean that they alone perform it. They coordinate it while all members contribute.

10. Keep the first release small

For a safe working MVP, complete these features first:

- Customer and provider registration
- Login with role-based access
- Provider profile
- Create and manage services
- Browse and search services
- Book a service
- Accept or reject bookings

- Cancel and complete bookings
- Submit reviews
- Basic admin dashboard
- AI category recommendation

After these work, add:

- Favourites
- Notifications
- Service images
- Provider availability
- Simulated payments
- AI review summary
- Advanced provider matching

Your user stories already mark registration, login, browsing, search, service publishing, booking, payment, reviews and admin functions as primary requirements.

The central rule for the team should be:

Each member owns one complete module from database to frontend, but every member must understand the authentication flow, database relationships, API communication and overall architecture for the viva.